GUIA UNICO PARA TIME DE DEPLOY (PROMPTS + GERACAO DE RELATORIOS)

Este guia foi feito para time leigo em prompt e IA. Objetivo: gerar relatorios JasperReports com seguranca, sem depender de conhecimento tecnico profundo.

---

0. Etapa 0 (obrigatoria): Verificar ambiente antes do primeiro relatorio

Antes de gerar qualquer relatorio, execute o script de verificacao de pre-requisitos:

Windows (PowerShell):

```
.\setup\check-env.ps1
```

Linux / Mac:

```
bash setup/check-env.sh
```

O script verifica: Java >= 11, Maven, Node.js >= 16 e variaveis de banco de dados (DB_URL, DB_USER, DB_PASSWORD). Corrija todos os erros reportados antes de continuar.

---

1. Qual modo escolher

Use SIMPLE quando houver 1 query e sem subreport. Use MASTER_DETAIL quando houver relacao pai-filho (1:N), com master e detail separados.

Regra pratica:

- Se o PDF tem uma tabela unica -> SIMPLE.
- Se o PDF tem bloco principal e lista filha por registro -> MASTER_DETAIL.
- Se o PDF tem bloco principal, lista filha e sub-lista aninhada -> MASTER_DETAIL_2L.

---

2. Fluxo obrigatorio (fail-safe)

Sempre siga esta ordem:

1. Validar se view, campos e relacionamento existem em `rules/views.json`.
2. Gerar JRXML(s) com parametros (`$P{...}`), sem valor hardcoded.
3. Validar XML/SQL com `node scripts/validate.js ...`.
4. Compilar com `node scripts/compile.js ... --pdf`.
5. Conferir artefatos finais (.jrxml, .jasper, .pdf, .log, metadata.json).

Se qualquer etapa falhar, corrigir e repetir antes de seguir.

---

3. Como pedir para a IA (roteiro simples)

Copie o exemplo do modo correto (SIMPLE ou MASTER_DETAIL), troque os valores e envie no chat.

Antes de preencher o prompt: confirme que a view e os campos desejados existem em `rules/views.json`. Se nao existirem, use o PASSO 0 do template (`prompts/relatorio-simples.prompt.md` ou `prompts/relatorio-master-detail.prompt.md`) para declarar os campos. A IA atualizara o `rules/views.json` como primeira acao antes de gerar o JRXML.

Boas praticas:

- Sempre informar view e campos explicitos.
- Sempre informar filtros com tipo (DATE, INT, STRING, DECIMAL).
- Nunca pedir SELECT *.
- Para MASTER_DETAIL, sempre informar chave de relacao e cardinalidade 1:N.

Regra de seguranca: campos informados sao imutaveis

> **O que acontece se voce informar um campo que nao existe?**

A IA segue esta logica obrigatoria para qualquer campo informado no prompt (filtro, campo de exibicao, chave de relacao):

Situacao	O que a IA faz
Campo existe em `rules/views.json`	Continua normalmente
Campo **nao** existe em `rules/views.json`, mas **existe** na view do banco	Adiciona o campo ao `rules/views.json` e continua
Campo **nao** existe em `rules/views.json` **e nem** na view do banco	**Bloqueia.** Exibe mensagem de erro com o nome exato do campo invalido e aguarda voce corrigir o prompt

Regra de ouro — nunca sera violada: A IA **nunca** substituira um campo que voce informou por outro campo, mesmo que sejam do mesmo tipo ou tenham nome parecido.

```
❌ PROIBIDO: trocar "data_inicio" por "data_criacao" porque ambos sao DATE
❌ PROIBIDO: usar "valor_bruto" no lugar de "valor_total" porque ambos sao DECIMAL
✅ CORRETO: bloquear e exibir mensagem de erro para voce corrigir
```

Se receber um erro de campo invalido, corrija o nome do campo no prompt e reenvie. A IA nunca vai adivinhar ou improvizar.

---

4. Prompt SIMPLE - opcoes, obrigatoriedade e descricao minima

Fonte de verdade do template: `prompts/relatorio-simples.prompt.md`.

- Nome do Relatorio: Obrigatorio.
  - O que preencher: nome tecnico curto e unico (ex.: VENDAS_DIARIAS_POR_VENDEDOR).
- Descricao: Obrigatorio.
  - O que preencher: objetivo funcional em 1 frase.
- Cenario (Contexto + Por Que): Obrigatorio.
  - O que preencher: publico, frequencia de uso e decisao suportada.
- View Selecionada: Obrigatorio.
  - O que preencher: nome da view exatamente como em `rules/views.json`.
- Agrupado Por (campo(s) dimensao): Obrigatorio.
  - O que preencher: campo(s) que definem granularidade principal.
- Justificativa do agrupamento: Obrigatorio.
  - O que preencher: motivo de negocio para esse agrupamento.
- Campos Desejados: Obrigatorio.
  - O que preencher: lista de campos validos da view, separados por virgula.
- Ordenado Por (campo(s)): Opcional.
  - O que preencher: campo(s) e direcao de ordenacao (ASC/DESC), ex.: `data DESC, vendedor_nome ASC`.
  - Regra: se o relatorio tiver agrupamento, o campo de agrupamento deve ser a primeira chave do ORDER BY para que os grupos fiquem contiguos.

- Filtros: Opcional.
  - O que preencher: Nome, Tipo, Obrigatorio (Sim/Nao), Default, Label e Valor de Teste (opcional).
  - **Valor de Teste**: valor real para ser passado via `--param` durante a geracao do relatorio. A IA usara esse valor ao compilar o PDF de preview. Formato igual ao tipo: DATE= `2024-01-01` , INT= `1001` , STRING= `João` , DECIMAL= `99.99` .
- Layout e Estilo: Obrigatorio.
  - O que preencher: orientacao, cabecalho, colunas, rodape.
- Usar Modelo Visual JRXML?: Opcional.
  - O que preencher: Sim/Nao.
- Caminho do Modelo: Opcional (obrigatorio se usar modelo).
  - O que preencher: caminho em `/tmp/...jrxml` .
- Descricao do Modelo: Opcional (recomendado se usar modelo).
  - O que preencher: quais elementos visuais devem ser reaproveitados.
- Arquivo Modelo Legado: Opcional (deprecated).
  - O que preencher: caminho de referencia visual em `examples/` .

Exemplo completo de prompt SIMPLE (compativel com o projeto)

```
Nome do Relatorio: VENDAS_DIARIAS_POR_VENDEDOR
Descricao: Relatorio operacional diario com vendas por item e vendedor.

Cenario (Contexto + Por Que):
Acompanhamento diario de vendas para gerente comercial validar performance por vendedor e categoria.
Uso: fechamento diario e analise de queda de faturamento.
Publico: gerencia comercial e coordenadores.

View Selecionada: view_vendas_diarias

Agrupado Por (campo(s) dimensao):
data, vendedor_nome

Justificativa (por que este agrupamento?):
Permite leitura por dia e por vendedor, facilitando comparacao operacional.

Campos Desejados (separe por virgula):
data, vendedor_id, vendedor_nome, item_codigo, item_nome, categoria, quantidade, valor_unitario, valor

Ordenado Por(campo(s)): data_compra
```

```
Filtro 1:
  Nome: dataInicio
  Tipo: DATE
  Obrigatorio: Nao
  Default: null
  Label: "Data Inicial"
  Valor de Teste: 2024-01-01

Filtro 2:
  Nome: dataFim
  Tipo: DATE
  Obrigatorio: Nao
  Default: null
  Label: "Data Final"
  Valor de Teste: 2024-12-31

Filtro 3:
  Nome: vendedorNome
  Tipo: STRING
  Obrigatorio: Nao
  Default: null
  Label: "Vendedor"
  Valor de Teste: João

Orientacao: Landscape
Estilo: Tabela com cabecalho forte e linhas alternadas.
Cabecalho: Titulo + periodo aplicado + total de registros.
Rodape: Pagina X de Y + data/hora de emissao.

Usar Modelo Visual JRXML? Nao
Arquivo Modelo: (em branco)
Descricao: (em branco)
```

---

5. Prompt MASTER_DETAIL / MASTER_DETAIL_2L - opcoes, obrigatoriedade e descricao minima

O pipeline suporta até 2 níveis de detail:

- **MASTER_DETAIL**: master → detail (1 nível)
- **MASTER_DETAIL_2L**: master → detail1 → detail2 (2 níveis)

Fonte de verdade do template: `prompts/relatorio-master-detail.prompt.md`.

- Nome do Relatorio: Obrigatorio.

    - O que preencher: nome tecnico unico do conjunto master/detail.
- Descricao: Obrigatorio.
    - O que preencher: objetivo funcional com visao pai-filho.
- Cenario Master/Detail: Obrigatorio.
    - O que preencher: contexto do bloco master e do bloco detail.
- View Master: Obrigatorio.
    - O que preencher: view pai valida em `rules/views.json`.
- Agrupado Por MASTER: Obrigatorio.
    - O que preencher: chave de identidade da linha master.
- Justificativa MASTER: Obrigatorio.
    - O que preencher: por que essa chave representa entidade pai.
- Campos Master: Obrigatorio.
    - O que preencher: campos da tabela principal (chave + totais/resumos).
- View Detail: Obrigatorio.
    - O que preencher: view filha valida em `rules/views.json`.
- Agrupado Por DETAIL: Obrigatorio.
    - O que preencher: granularidade da linha filha.
- Justificativa DETAIL: Obrigatorio.
    - O que preencher: por que essa granularidade representa o detalhe.
- Campos Detail: Obrigatorio.
    - O que preencher: campos transacionais do subreport.
- Chave Master, Chave Detail, Jointura e Cardinalidade: Obrigatorio.
    - O que preencher: contrato da relacao 1:N.
- Ordenado Por MASTER (campo(s)): Opcional.
    - O que preencher: campo(s) e direcao para a query master, ex.: `vendedor_nome ASC`.
- Ordenado Por DETAIL (campo(s)): Opcional.
    - O que preencher: campo(s) e direcao para a query detail, ex.: `data DESC, item_nome ASC`.
- Filtros Master: Opcional.

    - O que preencher: parametros do dataset pai (Nome, Tipo, Obrigatorio, Default, Label e Valor de Teste).
    - **Valor de Teste**: valor real para ser passado via `--param` ao gerar o PDF de preview. Mesmo formato do tipo: DATE= `2024-01-01` , INT= `1001` , STRING= `texto` , DECIMAL= `99.99` .
- Filtros Detail: Opcional.
    - O que preencher: parametros do dataset filho (mesmos campos do Filtros Master).
- Layout e Estilo: Obrigatorio.
    - O que preencher: como master e detail aparecem no PDF.

Exemplo completo de prompt MASTER_DETAIL (compativel com o projeto)

```
Nome do Relatorio: VENDEDOR_VENDAS_MASTER_DETAIL
Descricao: Relatorio de vendedores com historico detalhado de vendas por item.

Cenario Master/Detail:
MASTER (Principal): vendedor com indicadores consolidados.
DETAIL (Aninhado): vendas do vendedor por data e item.
USO: analise de desempenho semanal por carteira.
PUBLICO: gerencia comercial e lideres regionais.

View Master (Principal): view_vendas_diarias

Agrupado Por MASTER (chave identidade):
vendedor_id, vendedor_nome

Justificativa:
Cada linha master representa um vendedor unico por vendedor_id.

Campos Master (separe por virgula):
vendedor_id, vendedor_nome, categoria, SUM(valor_total) as faturamento, SUM(quantidade) as itens_vendi

Ordenado Por MASTER (campo(s)): vendedor_nome ASC

View Detail (Filha): view_vendas_diarias

Agrupado Por DETAIL (linhas dentro do detail):
data, item_nome, categoria

Justificativa:
Cada linha detail representa item vendido em data especifica do vendedor selecionado.

Campos Detail (separe por virgula):
vendedor_id, data, item_nome, item_codigo, categoria, quantidade, valor_unitario, valor_total
```

```
Ordenado Por DETAIL (campo(s)): data DESC, item_nome ASC

Chave Master: vendedor_id
Chave Detail: vendedor_id

SQL de Jointura (confirme a relacao):
Master.vendedor_id = Detail.vendedor_id

Cardinalidade Esperada:
1 vendedor : N vendas (1:N)

Filtro Master 1:
  Nome: dataInicio
  Tipo: DATE
  Obrigatorio: Nao
  Label: "Data Inicial"
  Valor de Teste: 2024-01-01

Filtro Master 2:
  Nome: dataFim
  Tipo: DATE
  Obrigatorio: Nao
  Label: "Data Final"
  Valor de Teste: 2024-12-31

Filtro Detail 1:
  Nome: valorMinimoVenda
  Tipo: DECIMAL
  Obrigatorio: Nao
  Default: null
  Label: "Valor Minimo"
  Valor de Teste: 50.00

Orientacao: Landscape
Tabela Master: vendedor e totais consolidados.
Tabela Detail: vendas associadas em grade filha.
Espacamento: detail abaixo do registro master.
Bordas: simples.
Cores: alternadas no master e neutras no detail.
```

6. Validacao de compatibilidade dos exemplos com os artefatos do projeto

Validacao realizada contra `rules/views.json`:

- OK SIMPLE: todos os campos do exemplo existem em `views.view_vendas_diarias.validFields`.
- OK MASTER: campos usados no exemplo existem em `views.view_vendas_diarias.validFields`.
- OK DETAIL: campos usados no exemplo existem em `views.view_vendas_diarias.validFields`.
- OK Relacao: chave `vendedores_vendas` existe em `relationships` com:
  - masterView: `view_vendas_diarias`
  - detailView: `view_vendas_diarias`
  - cardinality: `1:N`
  - localKey/foreignKey: `vendedor_id`

Relacao recomendada no compile para este exemplo:

```
node scripts/compile.js output/<nome>/master.jrxml --detail output/<nome>/detail.jrxml --relationship
```

---

7. Comandos operacionais prontos para uso

SIMPLE

```
node scripts/validate.js output/<nome>/relatorio.jrxml
node scripts/compile.js output/<nome>/relatorio.jrxml --pdf
```

Testar filtros com valores reais (opcional, mas recomendado)

Se o relatorio tem filtros, passe valores reais com `--param` para verificar se o PDF retorna dados. Use os **mesmos nomes de parametro** declarados no JRXML (ex.: `dataInicio`, `vendedorNome`).

```
# SIMPLE com filtros
node scripts/compile.js output/<nome>/relatorio.jrxml --pdf \
  --param dataInicio=2024-01-01 \
  --param dataFim=2024-12-31 \
  --param vendedorNome=João

# MASTER_DETAIL com filtros
node scripts/compile.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail.jrxml \
```

```
  --relationship vendedores_vendas --pdf \
  --param dataInicio=2024-01-01 \
  --param dataFim=2024-12-31

# Tipos aceitos: DATE=YYYY-MM-DD  INT=numero  STRING=texto  DECIMAL=99.99
```

> **Como saber os nomes dos filtros?** Abra o `.jrxml` do relatorio e procure pelas linhas `<parameter name="..."` .
> Os valores apos `name=` sao os nomes que voce deve usar apos `--param` .

MASTER_DETAIL

```
node scripts/validate.js output/<nome>/master.jrxml
node scripts/validate.js output/<nome>/detail.jrxml

node scripts/compile.js output/<nome>/master.jrxml --detail output/<nome>/detail.jrxml --relationship
```

MASTER_DETAIL_2L

```
node scripts/validate.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail1.jrxml \
  --detail2 output/<nome>/detail2.jrxml \
  --relationship <relKey>

node scripts/compile.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail1.jrxml \
  --detail2 output/<nome>/detail2.jrxml \
  --relationship <relKey> --pdf
```

Opcoes avancadas do validate.js

```
# Validacao com contaminacao de modelo (obrigatorio se usou modelo JRXML em /tmp)
node scripts/validate.js output/<nome>/relatorio.jrxml \
  --check-model-contamination /tmp/modelo.jrxml

# Validacao semantica MASTER_DETAIL (chaves, tipos e relacionamento)
node scripts/validate.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail.jrxml \
  --relationship vendedores_vendas

# Validacao semantica MASTER_DETAIL_2L
```

```
node scripts/validate.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail1.jrxml \
  --detail2 output/<nome>/detail2.jrxml \
  --relationship <relKey>
```

Gerar PDF avulso a partir de .jasper ja compilado

```
# Util quando o .jasper ja existe e voce quer apenas gerar novo PDF com outros parametros
node scripts/generate-pdf-with-data.js \
  output/<nome>/relatorio.jasper \
  output/<nome>/relatorio-novo.pdf \
  dataInicio=2024-01-01 dataFim=2024-12-31
```

Compilar com rastreabilidade de estilo (–style-blueprint)

```
# Usar somente quando o blueprint de estilo foi extraido de um modelo JRXML
node scripts/compile.js output/<nome>/relatorio.jrxml --pdf \
  --style-blueprint output/<nome>/style.json
# O metadata.json resultante incluira o campo styleSource com confianca e origem do estilo
```

---

8. O que deve existir no final (artefatos)

SIMPLE:

- relatorio.jrxml
- relatorio.jasper
- relatorio.pdf
- relatorio.log
- metadata.json

MASTER_DETAIL:

- master.jrxml
- detail.jrxml
- master.jasper
- detail.jasper
- master.pdf
- master.log
- metadata.json

MASTER_DETAIL_2L:

- master.jrxml
- detail1.jrxml
- detail2.jrxml
- master.jasper
- detail1.jasper
- detail2.jasper
- master.pdf
- master.log
- metadata.json

---

9. Checklist final para time leigo

- O prompt foi preenchido sem campos em branco obrigatorios.
- A view e os campos existem em `rules/views.json`.
- Os campos informados nos filtros existem na view (campo de filtro invalido bloqueia a geracao — ver Secao 3).
- Os filtros estao tipados corretamente.
- A validacao (`validate.js`) rodou sem erro.
- A compilacao (`compile.js --pdf`) gerou PDF e log sem ERROR.
- O PDF nao esta vazio e os dados esperados aparecem.
- **(Opcional, recomendado se o relatorio tem filtros)** Testar com valores reais usando `--param` (ver Secao 7 — "Testar filtros com valores reais") e confirmar que o PDF retorna dados corretos.

---

10. Como modificar um report depois de criado (sem quebrar setup)

Use este bloco quando precisar alterar um relatorio existente (regra ou visual).

10.1 Regras obrigatorias para alteracao segura

- Alterar somente arquivos do relatorio alvo em `output/<nome>/`.
- Manter compatibilidade com JasperReports 6.2.0.
- Nao alterar scripts, dependencias, setup, runner ou configuracoes globais.
- Nao alterar `rules/views.json` sem autorizacao explicita.
- Nao usar `SELECT *`.
- Nao hardcode valores de filtro em SQL.

- Sempre usar parametros (`$P{...}`) para filtros.
- Sempre executar validacao e compilacao apos mudanca.
- Em caso de erro, parar e corrigir antes de continuar.

10.2 Prompt modelo para alteracao de REGRA (dados/filtros/campos)

```
Sou do time de deploy e preciso ALTERAR um relatorio existente sem quebrar o setup.

Relatorio alvo:
- Tipo: SIMPLE ou MASTER_DETAIL
- Caminho: output/<NOME_RELATORIO>/
- Arquivo(s): <relatorio.jrxml> ou <master.jrxml + detail.jrxml>

Objetivo da alteracao de regra:
- [Descreva exatamente a regra nova, ex.: incluir filtro opcional categoria]
- [Descreva exatamente o que deve permanecer igual]

Escopo permitido:
- Alterar apenas os JRXMLs do relatorio alvo.

Escopo proibido:
- Nao alterar scripts, setup, dependencias, regras globais ou outros relatorios.
- Nao alterar rules/views.json sem autorizacao explicita.

Validacoes obrigatorias:
1) Conferir view/campos/chaves em rules/views.json
2) Rodar validate.js no(s) JRXML
3) Rodar compile.js com --pdf
4) Confirmar PDF nao vazio, log sem ERROR e metadata presente

Entrega esperada:
- Mostrar o que foi alterado
- Listar artefatos gerados
- Confirmar explicitamente que nenhum arquivo fora do escopo foi modificado
```

Exemplo rapido (regra) - SIMPLE:

```
Objetivo: adicionar filtro opcional categoria (STRING) no relatorio VENDAS_DIARIAS_POR_VENDEDOR.
Manter: layout atual, colunas atuais e ordenacao atual.
```

Exemplo rapido (regra) - MASTER_DETAIL:

```
Objetivo: adicionar filtro opcional valorMinimoVenda (DECIMAL) no detail.
```

```
Manter: relacao vendedor_id, estrutura master/detail e layout atual.
```

10.3 Prompt modelo para alteracao VISUAL (sem mudar regra)

```
Sou do time de deploy e preciso ALTERAR apenas o visual de um relatorio existente.

Relatorio alvo:
- Tipo: SIMPLE ou MASTER_DETAIL
- Caminho: output/<NOME_RELATORIO>/
- Arquivo(s): <relatorio.jrxml> ou <master.jrxml + detail.jrxml>

Objetivo visual:
- [Descreva ajustes visuais, ex.: largura de colunas, alinhamento, header, rodape]

Restricao critica:
- Nao alterar query, fields, parametros, filtros, relacionamento ou logica de negocio.

Escopo permitido:
- Somente estilos/layout no(s) JRXML(s) do relatorio alvo.

Execucao obrigatoria:
1) Rodar validate.js
2) Rodar compile.js --pdf
3) Confirmar PDF renderizado corretamente e sem ERROR em log

Entrega esperada:
- Lista objetiva dos ajustes visuais aplicados
- Confirmacao de que regras de dados nao foram alteradas
```

Exemplo rapido (visual):

```
Objetivo: aumentar largura da coluna item_nome, centralizar coluna quantidade e destacar cabecalho com
Manter: SQL, parametros e filtros exatamente como estao.
```

10.4 Comandos minimos apos qualquer alteracao

SIMPLE:

```
node scripts/validate.js output/<nome>/relatorio.jrxml
node scripts/compile.js output/<nome>/relatorio.jrxml --pdf
# Opcional: passar filtros para validar com dados reais
# node scripts/compile.js output/<nome>/relatorio.jrxml --pdf --param dataInicio=2024-01-01 --param da
```

MASTER_DETAIL:

```
node scripts/validate.js output/<nome>/master.jrxml
node scripts/validate.js output/<nome>/detail.jrxml
node scripts/compile.js output/<nome>/master.jrxml --detail output/<nome>/detail.jrxml --relationship
# Opcional: passar filtros
# node scripts/compile.js output/<nome>/master.jrxml --detail output/<nome>/detail.jrxml --relationshi
```

MASTER_DETAIL_2L:

```
node scripts/validate.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail1.jrxml \
  --detail2 output/<nome>/detail2.jrxml \
  --relationship <relKey>
node scripts/compile.js output/<nome>/master.jrxml \
  --detail output/<nome>/detail1.jrxml \
  --detail2 output/<nome>/detail2.jrxml \
  --relationship <relKey> --pdf
```

Se algum comando falhar, nao prossiga. Corrija e execute novamente.

---

11. Especificacao de layout por banda

Referencia de alturas recomendadas, objetos obrigatorios e variaveis corretas por banda.

Banda	Altura recomendada	Objetos obrigatorios	Observacao
`pageHeader`	60px	`staticText` (titulo), `textField` (data/hora), `<line>` no fundo	Usar `printWhenDetailOverflows="true"` para repetir em cada pagina
`columnHeader`	25px	Labels em negrito, `<line>` no fundo	—
`groupHeader`	20px	`textField` com expressao `"Label: " + $F{campo}`, largura = pageWidth - leftMargin - rightMargin	Altura minima do objeto textField: 16px
`groupFooter`	25px	`<line>` no topo, `textField` com	—

Banda	Altura recomendada	Objetos obrigatorios	Observacao
		`$V{<NomeGrupo>_COUNT}`	
`pageFooter`	30px	`<line>` no topo, total de registros, paginacao	Ver aviso abaixo

Variaveis de paginacao (erro comum)

```
$V{PAGE_NUMBER}  ->  pagina atual  (ex.: 1, 2, 3...)
$V{PAGE_COUNT}   ->  total de paginas do relatorio

Formato correto no rodape:
  "Pagina " + $V{PAGE_NUMBER} + " de " + $V{PAGE_COUNT}

Erro comum: usar PAGE_NUMBER nos dois lugares -> rodape exibe "Pagina 2 de 2" em vez de "Pagina 2 de 5
```

FIM.